

Implementation of a Lexical Analyzer Using C

Aim :

To write and implement a C program to perform lexical analysis and identify different tokens such as keywords, identifiers, constants, operators, and special symbols from a given C statement.

Algorithm:

1. Start the program.
2. Define the list of C keywords.
3. Read a C statement from the user.
4. Scan the input character by character.
5. Ignore white spaces.
6. If the character is an alphabet or `_`, extract the complete word.
7. Check whether the word is a keyword or an identifier.
8. If the character is a digit, extract the number and classify it as a constant.
9. Check and identify arithmetic operators.
10. Check and identify relational and assignment operators.
11. Check and identify special symbols.
12. If the character does not belong to any category, classify it as an unknown symbol.
13. Continue scanning until the end of the input.
14. Stop the program.

Pseudocode :

START

Define the list of C keywords

Read the input C statement

Set `i = 0`

WHILE `input[i]` is not end of string

 IF `input[i]` is a white space

```

    Increment i

ELSE IF input[i] is alphabet OR '_'
    Extract the complete word

    IF word is a keyword
        Display word as "Keyword"
    ELSE
        Display word as "Identifier"
    END IF

ELSE IF input[i] is a digit
    Extract the complete number
    Display number as "Constant"

ELSE IF input[i] is '+', '-', '*', '/', or '%'
    Display as "Arithmetic Operator"
    Increment i

ELSE IF input[i] is '=', '<', or '>'
    IF next character is '='
        Display as "Relational Operator"
        Increment i by 2
    ELSE
        Display as "Operator"
        Increment i
    END IF

ELSE IF input[i] is '(', ')', '{', '}', '[', ']',
    ';', or ','
    Display as "Special Symbol"
    Increment i

ELSE
    Display as "Unknown Symbol"
    Increment i
END IF

END WHILE
STOP

```

Program :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[])
{
    char keywords[][20] = {
        "auto", "break", "case", "char", "const",
        "continue", "default", "do", "double",
        "else", "enum", "extern", "float", "for",
        "goto", "if", "int", "long", "register",
        "return", "short", "signed", "sizeof",
        "static", "struct", "switch", "typedef",
        "union", "unsigned", "void", "volatile",
        "while"
    };

    int i;

    for (i = 0; i < 32; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
            return 1;
    }

    return 0;
}

int main()
{
    char input[1000];
    int i = 0;
```

```

printf("Enter a C statement:\n");
fgets(input, sizeof(input), stdin);

while (input[i] != '\0')
{
    /* Ignore white spaces */
    if (isspace(input[i]))
    {
        i++;
        continue;
    }

    /* Identifier or Keyword */
    if (isalpha(input[i]) || input[i] == '_')
    {
        char word[50];
        int j = 0;

        while (isalnum(input[i]) || input[i] == '_')
        {
            word[j++] = input[i++];
        }

        word[j] = '\0';

        if (isKeyword(word))
            printf("%s\t\tKeyword\n", word);
        else
            printf("%s\t\tIdentifier\n", word);
    }

    /* Number */
    else if (isdigit(input[i]))
    {

```

```

char number[50];
int j = 0;

while (isdigit(input[i]) || input[i] == '.')
{
    number[j++] = input[i++];
}

number[j] = '\0';

printf("%s\t\tConstant\n", number);
}

/* Arithmetic Operators */
else if (input[i] == '+' || input[i] == '-' ||
        input[i] == '*' || input[i] == '/' ||
        input[i] == '%')
{
    printf("%c\t\tArithmetic Operator\n", input[i]);
    i++;
}

/* Relational / Assignment Operators */
else if (input[i] == '=' || input[i] == '<' ||
        input[i] == '>')
{
    if (input[i + 1] == '=')
    {
        printf("%c%c\t\tRelational Operator\n",
            input[i], input[i + 1]);
        i += 2;
    }
    else
    {
        printf("%c\t\tOperator\n", input[i]);
    }
}

```

```

        i++;
    }
}

/* Special Symbols */
else if (input[i] == '(' || input[i] == ')' ||
        input[i] == '{' || input[i] == '}' ||
        input[i] == '[' || input[i] == ']' ||
        input[i] == ';' || input[i] == ',')
{
    printf("%c\t\tSpecial Symbol\n", input[i]);
    i++;
}

/* Unknown Symbol */
else
{
    printf("%c\t\tUnknown Symbol\n", input[i]);
    i++;
}
}

return 0;
}

```

Output :

```
C:\Users\SAIL\Desktop\LAB-0  X  +  v
Enter a C statement:
int a = 10 + 22
int          Keyword
a            Identifier
=            Operator
10           Constant
+            Arithmetic Operator
22           Constant

-----
Process exited after 14.41 seconds with return value 0
Press any key to continue . . . |
```

Result :

Thus, the Lexical Analyzer was successfully implemented in C to identify keywords, identifiers, constants, operators, and special symbols from the given C statement.